

Руководство по установке и использованию Poetry на Windows с PyCharm

Официальная документация проекта Poetry (<https://python-poetry.org/>)

Установка и удаление

Перед установкой убедитесь, что установлена версия python 3.9+.

Принудительно запустите PowerShell **от имени администратора**, иначе переменные пути Poetry не будут установлены в системе автоматически.

С помощью PowerShell выполните команду:

```
(Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | python -
```

Для проверки установки выполните команду:

```
poetry --version
```

Для удаления poetry выполните эту же команду с флагом **--uninstall**:

```
(Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | python - --uninstall
```

Управление окружением poetry

Команды для управления ПО poetry содержат ключ **self**. Эти команды предназначены для управления окружения **самого poetry**, а не окружения конкретного проекта.

Обновление poetry до последней версии:

```
poetry self update
```

Показывает библиотеки (пакеты) исполняемой среды Poetry, их версию и описание:

```
poetry self show
```

Показывает только установленные в ручную плагины:

```
poetry self show plugins
```

Для фиксирования зависимостей и их версий poetry с помощью lock-файла используйте команду:

```
poetry self lock
```

Для установки ранее закрепленных зависимостей, включая аддоны, с соответствующими версиями используйте команду:

```
poetry self install
```

Обновить окружение самого Poetry в соответствии с зафиксированными версиями пакетов (включая аддоны), которые необходимы для работы этой установки Poetry:

```
poetry self sync
```

Для удаления или добавления зависимостей используйте команды, **они автоматически изменяют lock-файл**:

```
poetry self add  
poetry self remove
```

Плагины

Установите плагин **poetry-plugin-export** для выгрузки зависимостей в текстовый файл для стандартного менеджера зависимостей pip.

Выполните команду:

```
poetry self add poetry-plugin-export
```

Для выгрузки зависимостей текущего проекта выполните команду:

```
poetry export -f requirements.txt --output requirements.txt --without-hashes  
--without-urls
```

Если в проекте используются плагины, например в pre-commit хуке автоматически будут выгружаться зависимости в requirements.txt, можно указать плагин как обязательную зависимость проекта:

```
[tool.poetry.requires-plugins]  
poetry-plugin-export = ">=1.8"
```

Документация плагина по ссылке на репозиторий github (<https://github.com/python-poetry/poetry-plugin-export>)

Конфигурирование настроек Poetry

Poetry можно настроить с помощью команды **config** или напрямую в файле **config.toml**, который будет автоматически создан при первом запуске этой команды. Этот файл обычно можно найти в одной из следующих директорий:

- **Linux:** \$XDG_CONFIG_HOME/pypoetry или ~/.config/pypoetry
- **Windows:** %APPDATA%\pypoetry

Чтобы вывести список текущей конфигурации используйте команду:

```
poetry config --list
```

Poetry также предоставляет возможность иметь настройки, специфичные для проекта, путем передачи опции **--local** команде **config**.

```
poetry config virtualenvs.create true --local
```

Ваша локальная конфигурация приложения Poetry хранится в файле **poetry.toml**, который отделен от **pyproject.toml**.

Если вы хотите увидеть значение конкретной настройки, вы можете передать ее имя команде **config**.

```
poetry config virtualenvs.path
```

Чтобы изменить или добавить новую настройку конфигурации, вы можете передать значение после имени настройки:

```
poetry config virtualenvs.path /path/to/cache/directory/virtualenvs
```

Если вы хотите удалить ранее установленную настройку, вы можете использовать опцию **--unset**. Настройка затем вернется к своему значению по умолчанию.

```
poetry config virtualenvs.path --unset
```

При создании виртуального окружения проекта с помощью Poetry через **PyCharm** используйте настройку создания **виртуального окружения внутри проекта** (рис. 1).

Рисунок 1 - Генерация нового виртуального окружения с помощью Poetry в PyCharm

Это позволит, при необходимости, без труда удалить виртуальное окружение и не складировать лишние окружения в корневой директории Poetry.

Среда разработки автоматически создаст файл **poetry.toml** в проекте с записью:

```
[virtualenvs]  
in-project = true
```

Настройка проекта

Новый проект

Для создания нового проекта выполните команду:

```
poetry new poetry-demo
```

Эта команда создаст директорию poetry-demo со следующим содержимым:

```
poetry-demo
├── pyproject.toml
├── README.md
├── src
│   ├── poetry_demo
│   │   └── __init__.py
├── tests
│   └── __init__.py
```

В отличие от других пакетов, **Poetry не устанавливает интерпретатор Python за вас** автоматически. Если вы хотите запускать Python-файлы в вашем пакете как скрипты или приложения, вы должны предоставить свой собственный интерпретатор Python для их запуска.

Poetry **потребует от вас явно указать, какие версии Python вы намерены поддерживать**, и его универсальный механизм блокировки гарантирует, что ваш проект можно установить (и все зависимости заявляют о поддержке) на всех заявленных версиях Python. Опять же, важно помнить, что – в отличие от других зависимостей – **указание версии Python лишь определяет, какие версии Python вы намерены поддерживать**. Пример:

```
[project]
requires-python = ">=3.9"
```

Разрешено устанавливать любую версию Python 3, которая больше или равна 3.9.0.

Когда вы запускаете **poetry install**, у вас **должен быть доступ к какой-либо версии интерпретатора Python, удовлетворяющей этому ограничению**, в вашей системе. **Poetry не установит интерпретатор Python за вас.**

Инициализация существующего проекта

Вместо создания нового проекта Poetry можно использовать для «инициализации» уже существующей директории. Чтобы интерактивно создать файл **pyproject.toml** в директории **pre-existing-project**:

```
cd pre-existing-project
poetry init
```

Режимы работы

Poetry может работать в двух разных режимах. Режим по умолчанию – **режим пакета (package mode)**. Это правильный режим, если вы хотите упаковать ваш проект в sdist или wheel и, возможно, опубликовать его в репозитории пакетов. В этом режиме метаданные, такие как **description authors readme build-system** являются обязательными. Кроме того, **сам проект будет установлен в редактируемом режиме при запуске poetry install**. Для установки только зависимостей в этом режиме нужно выполнять команду установки с флагом **poetry install –no-root**.

Если вы хотите использовать Poetry **только для управления зависимостями**, но не для упаковки, вы можете использовать **режим не-пакета (non-package mode)**.

Укажите в проекте раздел **tool.poetry** и соответствующий ключ со значением:

```
[tool.poetry]
package-mode = false
```

Удалите ключи **description authors readme** и раздел **build-system** целиком.

Файл **pyproject.toml** должен выглядеть, примерно, так:

```
[project]
name = "poetrytester"
version = "0.1.0"

requires-python = ">=3.12"

dependencies = [
]

[tool.poetry]
package-mode = false
```

В этом режиме невозможно собрать дистрибутив или опубликовать проект в репозитории пакетов. Кроме того, при запуске **poetry install** - Poetry не пытается установить сам проект, а только его зависимости (аналогично команде **poetry install –no-root** в пакетном режиме).

Установка зависимостей

Чтобы установить определенные зависимости для вашего проекта, просто запустите команду **install**.

```
poetry install
```

Когда вы запускаете эту команду, может произойти одно из двух:

Установка без poetry.lock

Если вы никогда раньше не запускали эту команду и файл **poetry.lock** также отсутствует, Poetry просто разрешает все зависимости, перечисленные в вашем файле **pyproject.toml**, и загружает последние версии их файлов. Когда Poetry завершает

установку, он записывает все пакеты и их точные версии, которые он загрузил, в файл **poetry.lock**, фиксируя проект на этих конкретных версиях. Вы должны закоммитить файл **poetry.lock** в репозиторий вашего проекта, чтобы все люди, работающие над проектом, использовали **одинаковые версии зависимостей**.

Установка с poetry.lock

Если при запуске `poetry install` уже присутствует файл **poetry.lock** вместе с файлом **pyproject.toml**, это означает, что либо вы уже запускали команду **install** ранее, либо кто-то другой в проекте запустил команду **install** и закоммитил файл **poetry.lock** в проект. В любом случае, запуск **install** при наличии файла **poetry.lock** разрешает и устанавливает все зависимости, которые вы перечислили в **pyproject.toml**, но Poetry использует точные версии, указанные в **poetry.lock**, чтобы гарантировать, что версии пакетов одинаковы для всех, кто работает над вашим проектом. В результате у вас будут все зависимости, запрошенные вашим файлом **pyproject.toml**, но они могут быть не самыми последними доступными версиями (некоторые зависимости, перечисленные в **poetry.lock**, могли выпустить новые версии после создания файла). Это сделано преднамеренно, это гарантирует, что ваш проект не сломается из-за неожиданных изменений в сторонних зависимостях.

Коммит файла poetry.lock в систему контроля версий

Разработчики приложений всегда коммитят **poetry.lock** в репозиторий, чтобы получать гарантированно воспроизводимые сборки.

Коммит этого файла в VCS важен, потому что это гарантирует то, что любой, кто настраивает проект, будет использовать точно те же версии зависимостей, которые используете вы. CI-сервер, производственные машины, другие разработчики в вашей команде – все и всё работает на одних и тех же зависимостях, что снижает вероятность появления ошибок, затрагивающих только некоторые части развертываний. Даже если вы разрабатываете в одиночку, через шесть месяцев при переустановке проекта вы можете быть уверены, что установленные зависимости все еще работают, даже если ваши зависимости выпустили много новых версий с тех пор.

Обновление зависимостей до их последних версий

Файл **poetry.lock** предотвращает автоматическое получение последних версий ваших зависимостей. Чтобы обновиться до последних версий, используйте команду **update**. Эта команда установит последние соответствующие версии (согласно вашему файлу **pyproject.toml**) и обновит файл блокировки новыми версиями. (Это эквивалентно удалению файла **poetry.lock** и повторному запуску **install**.)

Poetry будет отображать Предупреждение (Warning) при выполнении команды **install**, если **poetry.lock** и **pyproject.toml** не синхронизированы.

Управление зависимостями

Poetry поддерживает указание основных зависимостей в разделе **project** по ключу **dependencies** вашего **pyproject.toml**.

По историческим причинам и для определения дополнительной информации, которая используется только Poetry, можно использовать раздел **tool.poetry.dependencies**.

Примеры:

```
[project]
name = "poetrytester"
version = "0.1.0"

requires-python = ">=3.12"

dependencies = [
    "requests (>=2.32.5,<3.0.0)"
]
```

```
[project]
name = "poetrytester"
version = "0.1.0"

requires-python = ">=3.12"

[tool.poetry.dependencies]
requests = ">=2.32.5,<3.0.0"
```

Рекомендуется использовать первый вариант. Кроме прочего он будет явно выделять основную группу зависимостей.

Группы зависимостей

Poetry предоставляет способ организации ваших зависимостей по группам.

Зависимости, объявленные в **project.dependencies** (соответственно **tool.poetry.dependencies**), являются частью неявной основной (**main**) группы. Эти зависимости требуются вашему проекту во время выполнения.

Помимо основных зависимостей, у вас могут быть зависимости, которые нужны только для тестирования вашего проекта или для сборки документации.

Чтобы объявить новую группу зависимостей, используйте раздел **tool.poetry.group.<group>**, где **<group>** — это имя вашей группы зависимостей (например, **test**):

```
[tool.poetry.group.test.dependencies]
pytest = "^6.0.0"
pytest-mock = "*"
```

Все зависимости должны быть совместимы друг с другом без учета групп, поскольку они будут разрешены независимо от того, требуются они для установки или нет.

Думайте о группах зависимостей как о метках, связанных с вашими зависимостями: они не влияют на то, будут ли их зависимости разрешены и установлены по умолчанию, они просто представляют собой способ логически организовать зависимости.

Группы зависимостей, кроме неявной основной группы (**main**), должны содержать только зависимости, которые нужны вам в процессе разработки. Их установка возможна только с помощью Poetry.

Чтобы объявить набор зависимостей, которые добавляют дополнительную функциональность проекту во время выполнения, используйте вместо этого дополнения (extras). Дополнения могут быть установлены конечным пользователем с помощью `pip`.

Опциональные группы

Группа зависимостей может быть объявлена как опциональная. Это имеет смысл, когда у вас есть группа зависимостей, которые требуются только в определенном окружении или для конкретной цели.

```
[tool.poetry.group.docs]
optional = true

[tool.poetry.group.docs.dependencies]
mkdocs = "*"

```

Опциональные группы могут быть установлены в дополнение к зависимостям по умолчанию с помощью опции **—with** команды **install**.

```
poetry install --with docs

```

Добавление зависимости в группу

Команда **add** является предпочтительным способом добавления зависимостей в группу. Это делается с помощью опции **—group (-G)**.

```
poetry add pytest --group test

```

Если группа еще не существует, она будет создана автоматически.

Установка зависимостей группы

По умолчанию зависимости из всех не-опциональных групп будут установлены при выполнении **poetry install**.

Набор зависимостей по умолчанию для проекта включает неявную основную группу (main), а также все группы, которые не помечены явно как опциональные.

Вы можете исключить одну или несколько групп с помощью опции **—without**:

```
poetry install --without test,docs

```

Вы также можете подключить опциональные группы, используя опцию **—with**:

```
poetry install --with docs

```


При совместном использовании **—without** имеет приоритет над **—with**. Например, следующая команда установит **только зависимости, указанные в опциональной группе test**.

```
poetry install --with test,docs --without docs
```

Наконец, в некоторых случаях вы можете захотеть установить только определенные группы зависимостей без установки набора зависимостей по умолчанию. Для этой цели вы можете использовать опцию **—only**.

```
poetry install --only docs
```

Если вы хотите установить только зависимости времени выполнения проекта, вы можете сделать это с помощью обозначения **—only main**:

```
poetry install --only main
```

Удаление зависимостей из группы

Команда **remove** поддерживает опцию **—group** для удаления пакетов из определенной группы:

```
poetry remove mkdocs --group docs
```

Синхронизация зависимостей

Poetry поддерживает так называемую синхронизацию зависимостей. Синхронизация зависимостей гарантирует, что зафиксированные зависимости в файле **poetry.lock** — это единственные зависимости, присутствующие в окружении, удаляя всё, что не нужно.

Это делается с помощью команды **sync**:

```
poetry sync
```

Команда **sync** может быть объединена с любыми опциями, связанными с группами зависимостей, для синхронизации окружения с определенными группами. Обратите внимание, что дополнения (extras) учитываются отдельно. Любые дополнения, не выбранные для установки, всегда удаляются.

```
poetry sync --without dev
poetry sync --with docs
poetry sync --only dev
```

Команды

Вы уже узнали, как использовать интерфейс командной строки для выполнения некоторых действий. В этой главе будут приведены часто используемые и полезные команды.

Чтобы получить справку из командной строки, просто вызовите **poetry**, чтобы увидеть полный список команд, затем **—help** в сочетании с любой из них может дать вам более подробную информацию.

add

Команда **add** добавляет необходимые пакеты в ваш **pyproject.toml** и устанавливает их.

Если вы не укажете ограничение версии, poetry попытается использовать последнюю версию.

```
poetry add requests pendulum
```

Пакет по умолчанию ищется только в PyPI.

Вы также можете указать ограничение при добавлении пакета:

```
# Разрешить версии >=2.0.5, <3.0.0
poetry add pendulum@^2.0.5

# Разрешить версии >=2.0.5, <2.1.0
poetry add pendulum@~2.0.5

# Разрешить версии >=2.0.5, без верхней границы
poetry add "pendulum>=2.0.5"

# Разрешить только версию 2.0.5
poetry add pendulum==2.0.5
```

Если вы попытаетесь добавить пакет, который уже присутствует, вы получите ошибку. Однако, если вы укажете ограничение, как указано выше, зависимость будет обновлена с использованием указанного ограничения.

Если вы хотите получить последнюю версию уже присутствующей зависимости, вы можете использовать специальное ограничение **latest**:

```
poetry add pendulum@latest
```

Если пакет(ы), которые вы хотите установить, предоставляют дополнения (**extras**), вы можете указать их при добавлении пакета:

```
poetry add "requests[security,socks]"
```

Некоторые оболочки могут рассматривать квадратные скобки ([и]) как специальные символы. **Рекомендуется всегда заключать в кавычки аргументы**, содержащие эти символы, чтобы предотвратить неожиданное расширение оболочки.

Если вы хотите добавить пакет в определенную группу зависимостей, вы можете использовать опцию **—group (-G)**:

```
poetry add mkdocs --group docs
```

check

Команда **check** проверяет содержимое файла **pyproject.toml** и его соответствие файлу **poetry.lock**. Она возвращает подробный отчет, если есть какие-либо ошибки.

```
poetry check
```

env

Команда **env** группирует подкоманды для взаимодействия с виртуальными окружениями (**virtualenvs**), связанными с конкретным проектом.

```
env activate
```

Команда **env activate** выводит команду для активации виртуального окружения в вашей текущей оболочке.

Эта команда не активирует виртуальное окружение, а только отображает команду активации.

```
env info
```

Команда **env info** отображает информацию о текущем окружении.

Опции

- **—path (-p)**: Отображать только путь к окружению.
- **—executable (-e)**: Отображать только путь к исполняемому файлу python окружения.

```
env list
```

Команда **env list** выводит список всех виртуальных окружений, связанных с текущим проектом.

Опции

- **—full-path**: Выводить полные пути к виртуальным окружениям.

```
env remove
```

Команда **env remove** удаляет виртуальные окружения, связанные с проектом. Вы можете указать несколько исполняемых файлов Python или имен виртуальных окружений, чтобы удалить все соответствующие. В качестве альтернативы, вы можете удалить все связанные виртуальные окружения, используя опцию **—all**.

Если для конфигурации **virtualenvs.in-project** установлено значение **true**, аргументы или опции не требуются. Ваше внутрипроектное виртуальное окружение удаляется.

Аргументы

- **python**: Исполняемые файлы python, связанные с виртуальными окружениями, или имена виртуальных окружений, которые должны быть удалены. Можно указать несколько раз.

Опции

- **–all**: Удалить все управляемые виртуальные окружения, связанные с проектом.

```
env use
```

Команда **env use** активирует или создает новое виртуальное окружение для текущего проекта.

Аргументы

- **python**: Исполняемый файл python для использования. Это может быть номер версии (если не в Windows) или путь к бинарному файлу python.

lock

Эта команда блокирует (без установки) зависимости, указанные в **pyproject.toml**.

По умолчанию пакеты, которые уже были добавлены в файл блокировки ранее, не будут обновлены. Чтобы обновить все зависимости до последних доступных совместимых версий, используйте **poetry update –lock** или **poetry lock –regenerate**, которые обычно дают одинаковый результат.

```
poetry lock
```

Опции

- **–regenerate**: Игнорировать существующий файл блокировки и перезаписать его новым файлом блокировки, созданным с нуля.

python

Пространство имен python группирует подкоманды для управления версиями Python.

Это экспериментальная функция, и ее поведение может измениться в будущих выпусках.

```
python install
```

Команда **python install** устанавливает указанную версию Python из проекта Python Standalone Builds.

```
poetry python install <PYTHON_VERSION>
```

Опции

- **–reinstall**: Переустановить, если установка уже существует.

```
python list
```

Команда **python list** показывает версии Python, доступные в окружении. Это включает как установленные, так и обнаруженные системные управляемые и управляемые Poetry установки.

```
poetry python list
```

Опции

- `–all`: Показать все версии, включая те, что доступны для загрузки.
- `–implementation`: Реализация Python для поиска.
- `–managed`: Показать только версии Python, управляемые Poetry.

```
python remove
```

Команда `python remove` удаляет указанную версию Python, если она управляется Poetry.

```
poetry python remove <PYTHON_VERSION>
```

Опции

- `–implementation`: Реализация Python для использования. (`cpython`, `pyru`)

remove

Команда `remove` удаляет пакет из текущего списка установленных пакетов.

```
poetry remove pendulum
```

Если вы хотите удалить пакет из определенной группы зависимостей, вы можете использовать опцию **`–group (-G)`**:

```
poetry remove mkdocs --group docs
```

Опции

- `–group (-G)`: Группа, из которой нужно удалить зависимость.
- `–dev (-D)`: Удаляет пакет из зависимостей для разработки. (сокращение для `-G dev`)
- `–dry-run`: Выводит операции, но ничего не выполняет (неявно включает `–verbose`).
- `–lock`: Не выполнять операции (только обновить файл блокировки).

run

Команда **`run`** выполняет данную команду внутри виртуального окружения проекта.

```
poetry run python -V
```

Она также может выполнять один из скриптов, определенных в **`pyproject.toml`**.

Итак, если у вас определен скрипт так:

```
[project]
# ...
[project.scripts]
my-script = "my_module:main"
```

Вы можете выполнить его так:

```
poetry run my-script
```

Обратите внимание, что у этой команды нет опций.

update

Чтобы получить последние версии зависимостей и обновить файл **poetry.lock**, вы должны использовать команду **update**.

```
poetry update
```

Это разрешит все зависимости проекта и запишет точные версии в **poetry.lock**.

Если вы хотите обновить только несколько пакетов, а не все, вы можете перечислить их следующим образом:

```
poetry update requests
```

Обратите внимание, что это не будет обновлять версии зависимостей за пределами ограничений версий, указанных в файле **pyproject.toml**. Другими словами, **poetry update foo** будет бесполезен, если ограничение версии, указанное для foo, это ~2.3 или 2.3, а доступна 2.4. **Чтобы обновить foo, вы должны обновить ограничение, например, на ^2.3.** Вы можете сделать это с помощью команды **add**. Опции

- **–without:** Группы зависимостей, которые нужно игнорировать.
- **–with:** Опциональные группы зависимостей для включения.
- **–only:** Единственные группы зависимостей для включения.
- **–dry-run:** Выводит операции, но ничего не выполняет (неявно включает **–verbose**).
- **–lock:** Не выполнять установку (только обновить файл блокировки).
- **–sync:** Синхронизировать окружение с заблокированными пакетами и указанными группами.

Когда указан **–only**, опции **–with** и **–without** игнорируются.



Обсуждение

Лаврищук Владислав Михайлович, 2025/09/16 06:18 ()

Чет дохуя